

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

PROBLEM STATEMENT - AI-Powered Digital Twin Platform for Smart City Infrastructure

Industry Context

Smart cities increasingly adopt digital twin technology to simulate and monitor urban infrastructure in real time. AI, IoT, GIS, and cloud computing enable virtual replicas of roads, utilities, buildings, and transportation systems for predictive maintenance and strategic planning.

Problem Statement

Urban infrastructure management is challenging due to aging assets, increasing population, and limited real-time visibility. Traditional monitoring systems provide fragmented information and lack predictive capabilities. An AI-powered digital twin platform should create virtual models of city infrastructure by integrating real-time IoT data. The system should simulate infrastructure performance, detect anomalies, predict failures, and support data-driven urban planning. This solution will improve infrastructure reliability and maintenance efficiency.

Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning.

Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning decisions.
5. Accurate predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.

1. Introduction and Problem Analysis

1.1 Industry Context

Smart cities increasingly use Artificial Intelligence (AI), Internet of Things (IoT), Geographic Information Systems (GIS), and cloud computing to monitor and manage urban infrastructure. A Digital Twin is a virtual representation of a physical asset that continuously reflects its current condition using real-time data.

The proposed AI-Powered Digital Twin Platform for Smart City Infrastructure provides a centralized platform for monitoring roads, bridges, buildings, street lights and other urban infrastructure. IoT sensor data is collected and integrated into digital models of infrastructure assets. AI techniques are then used to identify abnormal conditions and predict possible infrastructure failures.

1.2 Problem Statement

Urban infrastructure management is difficult because of aging assets, increasing population, rising traffic and limited real-time visibility. Traditional monitoring systems often provide fragmented information and mainly support reactive maintenance.

The proposed system creates virtual models of city infrastructure and integrates real-time IoT data. It monitors asset health, detects anomalies, predicts possible failures and supports predictive maintenance. The platform can also simulate infrastructure scenarios to help authorities make better planning decisions.

1.3 Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT sensor data.
3. Predict infrastructure failures using AI.
4. Continuously monitor asset health.
5. Support predictive maintenance.
6. Simulate urban infrastructure scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning and operational efficiency.

1.4 Functional Requirements

- Register and manage infrastructure assets.
- Receive and process IoT sensor data.
- Display real-time infrastructure conditions.
- Maintain a digital twin for each asset.
- Detect abnormal sensor conditions.
- Calculate infrastructure health and failure risk.

1.5 Non-Functional Requirements

- Scalability: The system should support additional infrastructure assets.
- Reliability: Sensor and asset information should be stored consistently.
- Performance: Dashboard information should be available with minimal delay.
- Maintainability: Components should be modular and independently maintainable.
- Portability: Docker should allow the system to run consistently across environments.
- Security: Sensitive configuration values should be maintained using environment variables.
- Extensibility: New sensor types and infrastructure categories should be easy to add.

1.6 Expected Outcomes

The proposed platform is expected to provide enhanced infrastructure monitoring, reduced maintenance costs, improved asset reliability, better planning decisions, accurate predictive maintenance, increased operational efficiency, real-time visualization and support for sustainable city development.

2. System Architecture and Component Design

The proposed system uses a layered, service-oriented architecture. The major components are separated into frontend, backend, AI, database and IoT simulation services.

Architecture Components

1. React Dashboard

Provides the user interface for viewing infrastructure assets, sensor values, health status, predictions and reports.

2. FastAPI Backend

Acts as the main application layer. It provides REST APIs for asset management, sensor data, AI predictions, simulations and reports.

3. IoT Simulator

Generates simulated sensor readings such as temperature, vibration, traffic, load and power consumption. This represents real-time IoT devices.

4. AI Prediction Engine

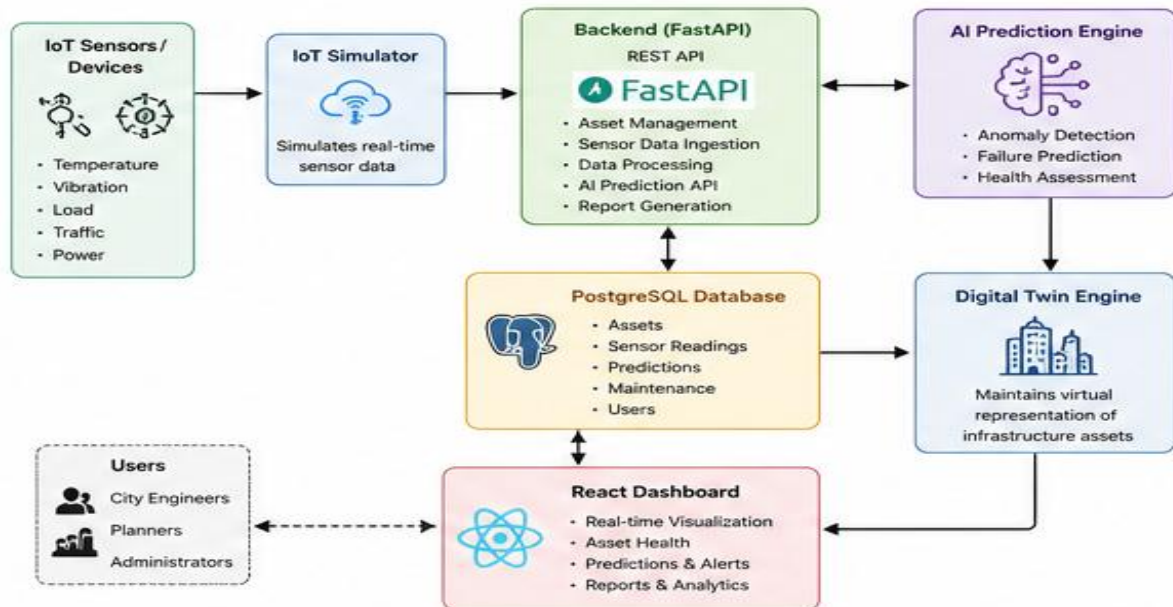
Processes sensor data and identifies abnormal conditions. It predicts infrastructure health and potential failure risk.

5. PostgreSQL Database

Stores infrastructure information, sensor readings, health status, predictions and maintenance information.

6. Digital Twin Engine

Maintains the virtual representation of each physical infrastructure asset using its latest sensor and health information.



The architecture follows a clear data flow:

IoT Sensors → Backend → Database/AI Engine → Digital Twin → Dashboard

The React frontend communicates with the FastAPI backend through REST APIs. The backend processes incoming IoT data, stores it in PostgreSQL and forwards relevant information to the AI prediction engine. The resulting health status and predictions are reflected in the corresponding digital twin and displayed on the dashboard.

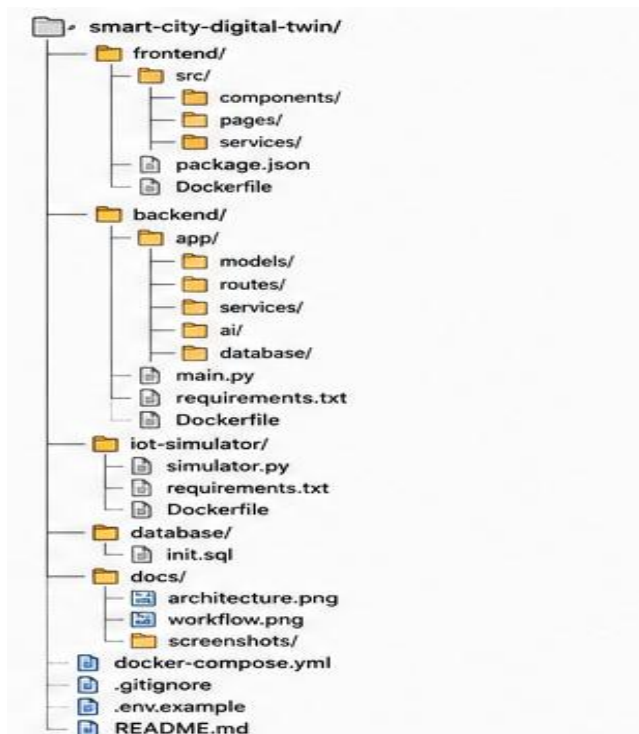
Architectural Justification

A layered and service-oriented architecture is suitable because each major function is separated into an independent component. This improves maintainability and allows individual components to be modified or scaled without redesigning the complete system.

Docker further improves portability by packaging each service with its required dependencies.

3. Project Structure and Component Responsibilities

The project repository is organized into separate directories for frontend, backend, IoT simulation, database and documentation.



Folder Justification

Folder/File	Purpose
frontend/	Contains the React dashboard
backend/	Contains FastAPI APIs and application logic
iot-simulator/	Generates simulated real-time sensor data
database/	Contains database initialization scripts
docs/	Stores diagrams, documentation and screenshots
Dockerfile	Defines the container environment
docker-compose.yml	Defines and connects application services
.gitignore	Prevents unnecessary or sensitive files from being committed
README.md	Provides project setup and usage instructions
.env.example	Documents required environment variables

Component Responsibilities

The frontend is responsible for visualization, the backend handles business logic and APIs, the IoT simulator generates sensor information, the AI engine performs prediction, and PostgreSQL provides persistent storage.

4. Git Repository and Branching Strategy

Git is used for version control and collaborative development. The repository maintains the complete history of source-code changes and enables safe development through branches.

Git Repository Initialization

```
git init
```

```
git add .
```

```
git commit -m "chore: initialize smart city digital twin project"
```

```
git branch -M main
```

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

Branching Strategy

The project follows a simplified Git Flow strategy:

main

|

└─ develop

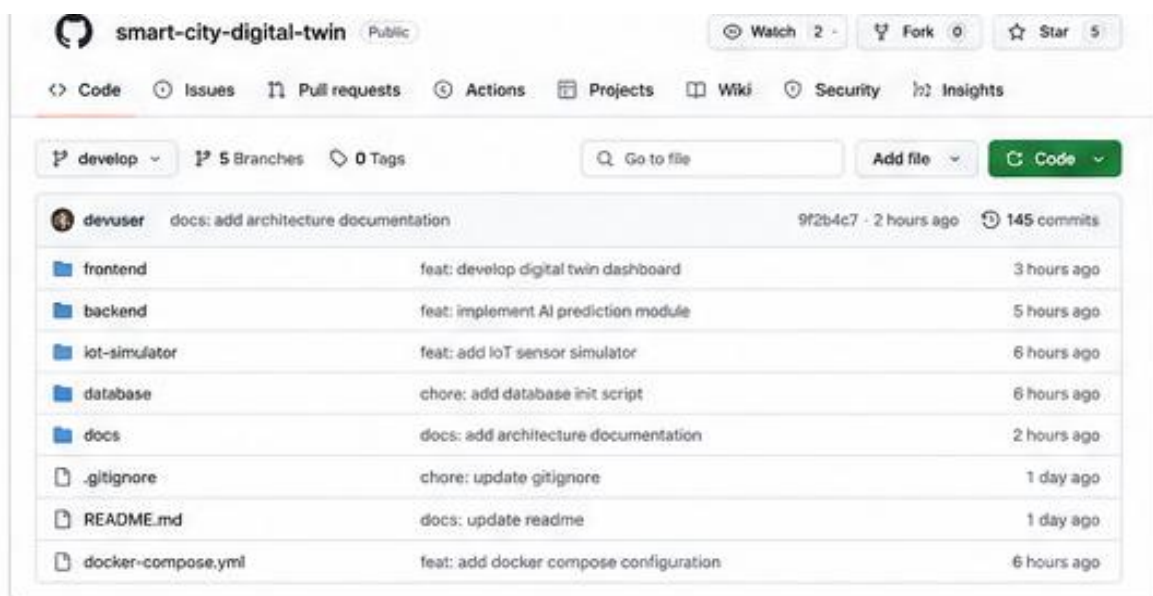
└─ feature/frontend-dashboard

└─ feature/backend-api

└─ feature/ai-prediction

└─ feature/iot-simulator

└─ feature/docker-deployment



The main branch contains stable versions. The develop branch integrates completed features. Feature branches are used for individual modules.

This strategy reduces the risk of unstable code reaching the main branch and provides an organized development process.

5. Version Control Workflow and Commit History

A meaningful Git workflow is followed throughout development.

Example Workflow

```
git checkout -b feature/backend-api
```

```
git add .
```

```
git commit -m "feat: implement infrastructure asset API"
```

```
git checkout develop
```

```
git merge feature/backend-api
```

```
git push origin develop
```

Meaningful Commit Messages

Example commits include:

```
chore: initialize project structure
```

```
feat: implement infrastructure asset API
```

```
feat: add IoT sensor simulator
```

```
feat: implement AI prediction module
```

```
feat: develop digital twin dashboard
```

```
feat: add Docker containerization
```

```
test: validate containerized application
```

```
docs: add architecture documentation
```

These messages clearly identify the purpose of each change and make the project history easier to understand and maintain.

Conflict Resolution

If two branches modify the same section of a file, Git identifies a merge conflict. The conflicting code is manually reviewed and corrected before executing:

```
git add .
```

```
git commit -m "fix: resolve merge conflict"
```

Repository Synchronization

Changes are synchronized with the remote repository using:

git pull origin develop

git push origin develop

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
smartcity-frontend-1	frontend	"npm start"	frontend	Up	0.0.0.0:3000->3000/tcp
smartcity-backend-1	backend	"uvicorn main:app ..."	backend	Up	0.0.0.0:8000->8000/tcp
smartcity-postgres-1	postgres:15	"docker-entrypoint.s..."	postgres	Up	0.0.0.0:5432->5432/tcp

8. Component Interaction and Quality Attributes

Component Interaction

The complete interaction is:

IoT Simulator



FastAPI Backend



PostgreSQL Database



AI Prediction Engine



Digital Twin State



React Dashboard



Maintenance / Planning Decision

The IoT simulator provides sensor measurements. The backend validates and processes the measurements and stores them in PostgreSQL. The AI engine analyses the sensor information and calculates the infrastructure health or failure risk. The digital twin is updated with the latest state, which is then displayed on the dashboard.

Quality Attribute Analysis

Scalability:

The modular service architecture allows additional infrastructure assets and services to be added.

Maintainability:

Frontend, backend, AI and IoT components are separated into independent modules.

Portability:

Docker packages the application and dependencies into portable containers.

Reliability:

Persistent database storage allows infrastructure and sensor information to be maintained.

Performance:

REST APIs provide structured communication between the dashboard and backend.

Security:

Environment variables can be used for database credentials and other configuration values.

Extensibility:

Additional sensor types, infrastructure categories and AI models can be integrated without redesigning the entire system.

Architectural Justification

The selected architecture is appropriate because the platform combines real-time sensor processing, AI analysis, persistent data storage and visualization. Separating these responsibilities reduces coupling between components and makes the system easier to test, maintain and extend. Docker Compose further simplifies deployment by managing the required services as a single application environment.

9. Benefits, Challenges, Future Enhancements and Conclusion

Benefits

- Real-time infrastructure monitoring.
- Early identification of abnormal conditions.
- Predictive maintenance support.
- Reduced dependency on manual inspection.
- Better infrastructure planning.
- Improved asset reliability.
- Centralized infrastructure information.
- Portable and reproducible deployment using Docker.
- Better collaboration and version management using Git.

Challenges

The major implementation challenges include integrating multiple services, maintaining communication between containers, generating realistic IoT data, preparing suitable data for AI prediction and maintaining consistency between physical infrastructure information and its digital representation.

Future Enhancements

Future versions can include:

1. Integration with real IoT hardware.
2. GIS-based city maps.
3. 3D infrastructure visualization.
4. Cloud deployment.
5. More advanced machine-learning models.
6. Real-time notification and alert systems.
7. Integration with traffic management systems.
8. Automated maintenance scheduling.
9. Historical trend analysis.
10. Large-scale smart city infrastructure monitoring.

Conclusion

The AI-Powered Digital Twin Platform for Smart City Infrastructure provides a unified approach to monitoring and managing urban infrastructure. By combining Digital Twin technology, IoT data, AI-based prediction, database management and visualization, the system can identify abnormal infrastructure conditions and support predictive maintenance.

The use of a modular software architecture improves maintainability and extensibility, while Git provides systematic version control and Docker enables portable and consistent deployment. The proposed platform therefore provides a practical foundation for intelligent, reliable and sustainable smart city infrastructure management.